Studio Nx

백엔드 213 — 패턴 스키마

2026-10-17

Backend(FastAPI/Python) 213 — 패턴 스키마 (`NoteInput` · `NoteItem` · `NotesSaveRequest` · `PatternUpdateRequest` · `PatternResponse` · `PatternListResponse`)

항목	내용
파일	`studionx/backend/app/schemas/pattern.py`
작업	S1-08 아코디언·피아노롤: 패턴 요청·응답 모양 (PRD 02 FR-231 ~ FR-233)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	212 `MidiNote` · 214 `pattern_service` 조회

1. 하는 일

패턴 API 가 주고받는 JSON 모양을 정합니다. 노트를 **통째로** 주고받으므로(215) 노트 하나의 모양(`NoteInput`)과 여러 개를 담는 요청(`NotesSaveRequest`)을 나눕니다. 칸 이름은 DB(`midi_notes`)와 피아노롤(`sequencer-roll.js`)이 쓰는 이름을 그대로 맞춥니다.

스키마	쓰는 곳
`NoteInput`	노트 한 건(보낼 때). `note_id` 는 받지 않습니다
`NoteItem`	노트 한 건(응답). `NoteInput` 에 `note_id` 를 더한 모양
`NotesSaveRequest`	`PUT /patterns/{id}/notes` — 노트 목록
`PatternUpdateRequest`	`PATCH /patterns/{id}` — 이름·색·길이·스윙
`PatternResponse`	목록·상세·저장 응답이 모두 쓰는 패턴 한 건
`PatternListResponse`	`GET /patterns` — 패턴 목록

2. 언제 쓰나

쓰는 곳	
`api/v1/patterns.py` (217)	요청 본문·응답 모델로 지정합니다
`pattern_service.to_response` (214)	`PatternResponse` 를 만듭니다
`pattern_service.replace_notes` (215)	`NotesSaveRequest.notes` 를 새 행으로 옮깁니다

3. 코드

칸 길이·개수 상한과 타입 별칭

```
PATTERN_NAME_MAX = 100          # patterns.name 칸 길이와 같음
PATTERN_COLOR_MAX = 7           # '#RRGGBB'
NOTES_MAX = 5000                # 실수로 큰 목록을 보내는 것 방지

PatternName = Annotated[str, StringConstraints(strip_whitespace=True, min_length=1, max_length=PATTERN_N
AME_MAX)]
Pitch = Annotated[int, Field(ge=0, le=127)]          # MIDI 0~127
```

```
Velocity = Annotated[int, Field(ge=1, le=127)]        # 1~127 (DB CHECK)
StartTick = Annotated[int, Field(ge=0)]
LengthTick = Annotated[int, Field(ge=1)]
Channel = Annotated[int, Field(ge=0, le=15)]
```

노트 한 건과 목록

```
class NoteInput(BaseModel):
    pitch: Pitch
    start_tick: StartTick
    length_tick: LengthTick
    velocity: Velocity = 100
    channel: Channel = 0
    release_velocity: Annotated[int, Field(ge=0, le=127)] = 64
    is_muted: bool = False
    probability: Decimal = Field(default=Decimal("1.00"), ge=0, le=1, decimal_places=2)
    velocity_deviation: Annotated[int, Field(ge=0, le=127)] = 0

class NoteItem(NoteInput):
    note_id: int

class NotesSaveRequest(BaseModel):
    notes: list[NoteInput] = Field(default_factory=list, max_length=NOTES_MAX)
```

패턴 수정 · 응답

```
class PatternUpdateRequest(BaseModel):
    name: PatternName | None = None
    color: Annotated[str, StringConstraints(strip_whitespace=True, max_length=PATTERN_COLOR_MAX)] | None
= None
    length_tick: Annotated[int, Field(ge=1)] | None = None
    swing: Annotated[Decimal, Field(ge=0, le=1, decimal_places=2)] | None = None

    @field_validator("name")
    @classmethod
    def _name_not_null(cls, value):
        if value is None:
            raise ValueError("name cannot be empty")
        return value

class PatternResponse(BaseModel):
    pattern_id: int
    project_id: int
    name: str
    color: str | None
    length_tick: int
    swing: Decimal
    updated_at: datetime
    notes: list[NoteItem]

class PatternListResponse(BaseModel):
    items: list[PatternResponse]
    total: int
```

문법 풀이

식	뜻
`Annotated[..., Field(ge=…, le=...)]`	값 범위를 붙입니다. 벗어나면 422 로 거부됩니다
`StringConstraints(strip_whitespace=True, ...)`	앞뒤 공백을 먼저 지우고 길이를 잽니다
`Field(max_length=NOTES_MAX)`	목록 길이 상한입니다. 한 패턴 5000개를 넘기면 422 입니다
`Decimal` + `decimal_places=2`	소수 둘째 자리까지 받습니다(`swing`, `probability`)
`Field(default_factory=list)`	요청에 `notes` 가 없으면 빈 목록으로 봅니다
`@field_validator`	값 하나를 더 검사합니다. 보낸 경우에만 돕니다

4. 왜 이렇게

선택	이유
이름을 화면·DB 와 동일하게	`start_tick`·`length_tick`·`pitch`·`velocity`. 피아노롤이 그대로 보내고 그대로 받습니다. 변환 코드가 없습니다
`NOTES_MAX = 5000`	잘못된 화면이 거대한 목록을 보내 서버를 힘들게 하는 것을 막습니다
`note_id` 는 응답에만	노트는 통째 교체라 새 번호가 매겨집니다. 보낼 때 번호를 받아도 쓸 데가 없습니다
`name` 에 null 금지 검사	`PATCH` 는 보낸 칸만 바꾸므로, `name: null` 을 직접 보낸 경우만 막습니다. 안 보내면 검사하지 않습니다

4. 시험

`tests/test_pattern_routes.py` 에서 422(빈 이름·음높이 128·길이 0·채널 16)와 정상 응답 모양을 확인했습니다. 통과합니다.